

□ ДОМЕН 1: АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ — ПОЛНАЯ ДЕТАЛИЗАЦИЯ

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 1.1: МАССИВЫ И СТРОКИ (JUNIOR → SENIOR)

1.1.1. Массивы (Junior Level)

№ 1.1.1.1 — Статические массивы

- **Что нужно знать:** объявление, индексация, размер фиксирован, память в стеке
- **Вопросы на собеседовании:** "Как объявить массив из 10 целых чисел в C++?"
- **Практика:** Написать программу, которая заполняет массив числами от 1 до 10 и выводит их в обратном порядке

№ 1.1.1.2 — Динамические массивы

- **Что нужно знать:** `new[]/delete[]` (C++), `std::vector` (C++), `List<T>` (C#), управление памятью
- **Вопросы на собеседовании:** "В чем разница между массивом в стеке и массивом в куче?"
- **Практика:** Реализовать класс `DynamicArray` с методами `push_back`, `pop_back`, `resize`

№ 1.1.1.3 — Амортизированная сложность

- **Что нужно знать:** как работает расширение вектора (обычно $\times 2$), амортизированная $O(1)$ для `push_back`
- **Вопросы на собеседовании:** "Почему вставка в конец вектора — $O(1)$ амортизированно?"
- **Практика:** Написать бенчмарк, который измеряет время вставки 1 млн элементов в `std::vector`

№ 1.1.1.4 — Базовые алгоритмы

- **Что нужно знать:** линейный поиск ($O(n)$), бинарный поиск ($O(\log n)$) на отсортированном массиве
- **Вопросы на собеседовании:** "Как найти элемент в отсортированном массиве быстрее $O(n)$?"
- **Практика:** Реализовать итеративный и рекурсивный бинарный поиск

№ 1.1.1.5 — Двумерные массивы

- **Что нужно знать:** представление (массив массивов, одномерный с индексацией), обход по строкам/столбцам
- **Вопросы на собеседовании:** "В чем разница между обходом матрицы по строкам и по столбцам (с точки зрения кэша)?"
- **Практика:** Написать программу для умножения матриц и сравнить производительность разных способов обхода

№ 1.1.1.6 — Сложность памяти

- **Что нужно знать:** $O(1)$ vs $O(n)$, работа с большими массивами (cache miss, TLB)
- **Вопросы на собеседовании:** "Почему обход матрицы по строкам быстрее, чем по столбцам?"
- **Практика:** Написать бенчмарк: обойти матрицу 10000×10000 по строкам и по столбцам, замерить время

1.1.2. Строки (Junior → Middle Level)

№ 1.1.2.1 — C-style строки

- **Что нужно знать:** `char*`, завершающий нуль (`\0`), функции `strlen`, `strcpy`, `strcat`, `strcmp`, опасность переполнения буфера
- **Вопросы на собеседовании:** "Как работает `strlen`? В чем проблема с `strcpy`?"
- **Практика:** Реализовать свои версии `strlen`, `strcpy`, `strcat` без использования стандартных функций

№ 1.1.2.2 — `std::string` (C++)

- **Что нужно знать:** внутреннее устройство (SSO — Small String Optimization), копирование при передаче, `c_str()`, `data()`
- **Вопросы на собеседовании:** "Как `std::string` оптимизирует хранение коротких строк?"

- **Практика:** Написать программу, показывающую разницу в адресах для короткой и длинной строки

№ 1.1.2.3 — String в C#

- **Что нужно знать:** неизменяемость (immutable), StringBuilder для эффективной конкатенации, интернирование
- **Вопросы на собеседовании:** "Почему в C# строки неизменяемы? Когда использовать StringBuilder?"
- **Практика:** Написать бенчмарк для конкатенации 10000 строк через + и через StringBuilder

№ 1.1.2.4 — Поиск подстроки

- **Что нужно знать:** наивный $O(n*m)$, КМП (Кнут-Моррис-Пратт) $O(n+m)$, алгоритм Бойера-Мура, Z-функция
- **Вопросы на собеседовании:** "Как найти все вхождения подстроки в строке?"
- **Практика:** Реализовать поиск подстроки через КМП и сравнить с наивным на больших данных

№ 1.1.2.5 — Хеширование строк

- **Что нужно знать:** полиномиальный хеш (Rabin-Karp), коллизии, использование для поиска
- **Вопросы на собеседовании:** "Как проверить, являются ли две строки анаграммами?"
- **Практика:** Реализовать алгоритм Рабина-Карпа для поиска подстроки

№ 1.1.2.6 — Регулярные выражения

- **Что нужно знать:** базовые конструкции ($.$, $*$, $+$, $?$, $[a-z]$, $\backslash d$), greedy vs lazy, группы
- **Вопросы на собеседовании:** "Что такое greedy и lazy квантификаторы в регулярках?"
- **Практика:** Написать регулярное выражение для валидации email-адреса

1.1.3. Сложные алгоритмы на массивах (Senior Level)

№ 1.1.3.1 — Два указателя (Two Pointers)

- **Что нужно знать:** техника для работы с отсортированными массивами (слияние, поиск пары)
- **Вопросы на собеседовании:** "Как найти все пары в отсортированном массиве, сумма которых равна X ?"
- **Практика:** Реализовать поиск пары с суммой X за $O(n)$

№ 1.1.3.2 — Скользящее окно (Sliding Window)

- **Что нужно знать:** фиксированное и динамическое окно, задачи (подстрока без повторений, сумма подмассива)
- **Вопросы на собеседовании:** "Как найти самую длинную подстроку без повторяющихся символов?"
- **Практика:** Реализовать алгоритм скользящего окна для задачи «подстрока без повторений»

№ 1.1.3.3 — Divide and Conquer

- **Что нужно знать:** быстрая сортировка (QuickSort), сортировка слиянием (MergeSort), поиск медианы
- **Вопросы на собеседовании:** "Как работает быстрая сортировка? В чем худший случай?"
- **Практика:** Реализовать быструю сортировку с выбором опорного элемента (медиана из трех)

№ 1.1.3.4 — Prefix Sum

- **Что нужно знать:** построение массива префиксных сумм, использование для запросов суммы подмассива за $O(1)$
- **Вопросы на собеседовании:** "Как быстро отвечать на запросы суммы на отрезке?"
- **Практика:** Реализовать класс PrefixSum с методами build и query

№ 1.1.3.5 — Difference Array

- **Что нужно знать:** техника для быстрого применения операций на отрезке
- **Вопросы на собеседовании:** "Как добавить число X ко всем элементам на отрезке $[L, R]$ за $O(1)$?"
- **Практика:** Реализовать разностный массив для обработки множественных запросов обновления

№ 1.1.3.6 — Битовая манипуляция

- **Что нужно знать:** побитовые операции (&, |, ^, ~, <<, >>), проверка четности, установка/сброс бита
 - **Вопросы на собеседовании:** "Как проверить, является ли число степенью двойки, без циклов?"
 - **Практика:** Написать функции для работы с битами: set_bit, clear_bit, toggle_bit, is_power_of_two
-

1.1.4. Практические задания (сквозные)

№ 1.1.4.1

- **Задание:** Реализовать свой std::vector с методами push_back, pop_back, insert, erase, итераторами, аллокатором
- **Что проверяет:** Понимание управления памятью и амортизированной сложности

№ 1.1.4.2

- **Задание:** Реализовать алгоритм КМП (поиск подстроки) и проверить его на тексте размером 10 млн символов
- **Что проверяет:** Понимание линейных алгоритмов на строках

№ 1.1.4.3

- **Задание:** Написать программу, которая находит максимальную сумму подмассива (алгоритм Кадане)
- **Что проверяет:** Знание динамического программирования

№ 1.1.4.4

- **Задание:** Реализовать свою версию std::sort (быстрая сортировка) и сравнить со стандартной
- **Что проверяет:** Понимание алгоритмов сортировки и их оптимизации

№ 1.1.4.5

- **Задание:** Написать конвертер римских чисел в арабские и обратно
 - **Что проверяет:** Работа со строками и отображениями
-

РАЗДЕЛ 1.2: СВЯЗНЫЕ СПИСКИ (JUNIOR → SENIOR)

1.2.1. Односвязные списки (Junior Level)

№ 1.2.1.1 — Структура узла

- **Что нужно знать:** `struct Node { int data; Node* next; }`, операции вставки/удаления в начало ($O(1)$) и в конец ($O(n)$)
- **Вопросы на собеседовании:** "Как выглядит узел односвязного списка?"
- **Практика:** Реализовать класс `SinglyLinkedList` с методами `push_front`, `push_back`, `pop_front`, `pop_back`

№ 1.2.1.2 — Итераторы

- **Что нужно знать:** обход списка с указателя `head`, цикл `while (curr != nullptr)`
- **Вопросы на собеседовании:** "Как найти середину односвязного списка?"
- **Практика:** Реализовать функцию для поиска среднего элемента списка

№ 1.2.1.3 — Разворот списка

- **Что нужно знать:** итеративно (меняем ссылки) и рекурсивно
- **Вопросы на собеседовании:** "Как развернуть односвязный список?"
- **Практика:** Реализовать оба способа: итеративный и рекурсивный разворот

№ 1.2.1.4 — Циклические списки

- **Что нужно знать:** как обнаружить цикл (алгоритм Флойда — черепаха и заяц)
- **Вопросы на собеседовании:** "Как определить, есть ли цикл в односвязном списке?"
- **Практика:** Реализовать алгоритм Флойда для поиска цикла и нахождения начала цикла

№ 1.2.1.5 — Проблемы

- **Что нужно знать:** потеря ссылок при удалении (утечки памяти), работа с `nullptr`
- **Вопросы на собеседовании:** "Как удалить узел из списка по значению?"
- **Практика:** Реализовать метод `remove(value)` с корректным удалением узла

1.2.2. Двусвязные списки (Middle Level)

№ 1.2.2.1 — Структура узла

- **Что нужно знать:** `struct Node { int data; Node* next; Node* prev; }`, вставка/удаление за $O(1)$
- **Вопросы на собеседовании:** "В чем преимущество двусвязного списка перед односвязным?"
- **Практика:** Реализовать класс `DoublyLinkedList` с методами `insert_before`, `insert_after`, `remove`

№ 1.2.2.2 — Память

- **Что нужно знать:** больше памяти на указатель, лучше для удаления последнего элемента
- **Вопросы на собеседовании:** "Как удалить узел из двусвязного списка, имея только указатель на этот узел?"
- **Практика:** Реализовать удаление узла за $O(1)$ по указателю

№ 1.2.2.3 — Реализация deque

- **Что нужно знать:** двусвязный список как основа для двусторонней очереди
- **Вопросы на собеседовании:** "Как реализовать стек и очередь на основе списка?"
- **Практика:** Реализовать стек и очередь на двусвязном списке

№ 1.2.2.4 — LRU Cache

- **Что нужно знать:** использование двусвязного списка + хеш-таблицы
- **Вопросы на собеседовании:** "Как реализовать LRU кеш?"
- **Практика:** Реализовать LRU Cache на C++ с использованием `unordered_map` и двусвязного списка

1.2.3. Экспертный уровень (Senior / Staff Level)

№ 1.2.3.1 — Skip List (список с пропусками)

- **Что нужно знать:** структура для быстрого поиска ($O(\log n)$), альтернатива сбалансированным деревьям

- **Вопросы на собеседовании:** "Как реализовать структуру данных с быстрым поиском и вставкой без использования деревьев?"
- **Практика:** Реализовать Skip List с поиском, вставкой и удалением

№ 1.2.3.2 — Lock-free списки

- **Что нужно знать:** структуры для многопоточного доступа без блокировок
- **Вопросы на собеседовании:** "Как реализовать потокобезопасный список без использования мьютексов?"
- **Практика:** Изучить алгоритм lock-free стека/списка (Michael-Scott)

№ 1.2.3.3 — Memory pool для списков

- **Что нужно знать:** выделение узлов из пула для улучшения производительности
- **Вопросы на собеседовании:** "Как ускорить работу с часто создаваемыми/удаляемыми узлами списка?"
- **Практика:** Реализовать свой allocator для узлов списка

№ 1.2.3.4 — Intrusive lists

- **Что нужно знать:** список, где узлы сами содержат ссылки (embedded pointers)
- **Вопросы на собеседовании:** "В чем отличие intrusive list от обычного?"
- **Практика:** Реализовать intrusive связный список для C++ класса

1.2.4. Практические задания (сквозные)

№ 1.2.4.1

- **Задание:** Реализовать LinkedList<T> на C++ и C# с полным набором методов
- **Что проверяет:** Знание синтаксиса шаблонов/дженериков и работы с памятью

№ 1.2.4.2

- **Задание:** Написать функцию для слияния двух отсортированных списков (merge)

- **Что проверяет:** Понимание работы с указателями и алгоритмов слияния

№ 1.2.4.3

- **Задание:** Реализовать алгоритм "удалить N-ый узел с конца списка" за один проход
- **Что проверяет:** Умение использовать два указателя

№ 1.2.4.4

- **Задание:** Реализовать разворот списка группами по K узлов
- **Что проверяет:** Сложная задача на перестановку ссылок

№ 1.2.4.5

- **Задание:** Реализовать LRU Cache с использованием двусвязного списка и хеш-таблицы
- **Что проверяет:** Комбинация двух структур данных

РАЗДЕЛ 1.3: СТЕК И ОЧЕРЕДЬ (JUNIOR → SENIOR)

1.3.1. Стек (Junior Level)

№ 1.3.1.1 — Определение

- **Что нужно знать:** LIFO (Last In, First Out), операции push ($O(1)$), pop ($O(1)$), top ($O(1)$), empty
- **Вопросы на собеседовании:** "Что такое стек? Приведи пример из реальной жизни"
- **Практика:** Реализовать стек на массиве и на списке

№ 1.3.1.2 — Применение

- **Что нужно знать:** проверка скобок, обратная польская запись, стек вызовов функций, Undo/Redo
- **Вопросы на собеседовании:** "Как проверить правильность расстановки скобок в выражении?"
- **Практика:** Написать программу для проверки сбалансированности скобок `()`, `{}`, `[]`

№ 1.3.1.3 — Реализация

- **Что нужно знать:** статический стек (массив фиксированного размера) и динамический (вектор)
- **Вопросы на собеседовании:** "Как реализовать стек с помощью `std::vector`?"
- **Практика:** Реализовать класс `Stack<T>` на основе `std::vector`

№ 1.3.1.4 — Two-stacks queue

- **Что нужно знать:** реализация очереди с помощью двух стеков
- **Вопросы на собеседовании:** "Как реализовать очередь, используя два стека?"
- **Практика:** Реализовать очередь с методами `enqueue` и `dequeue` на двух стеках (амортизированная $O(1)$)

№ 1.3.1.5 — Min stack

- **Что нужно знать:** стек с возможностью получения минимального элемента за $O(1)$
- **Вопросы на собеседовании:** "Как реализовать стек, в котором можно получить минимум за $O(1)$?"
- **Практика:** Реализовать стек с `get_min()` за $O(1)$, используя дополнительный стек

1.3.2. Очередь (Junior → Middle Level)

№ 1.3.2.1 — Определение

- **Что нужно знать:** FIFO (First In, First Out), операции `push` ($O(1)$), `pop` ($O(1)$), `front` ($O(1)$), `back` ($O(1)$)
- **Вопросы на собеседовании:** "Что такое очередь? Приведи пример"
- **Практика:** Реализовать очередь на массиве (кольцевая) и на списке

№ 1.3.2.2 — Кольцевая очередь

- **Что нужно знать:** использование фиксированного массива с `head` и `tail`, `mod` для заикливания
- **Вопросы на собеседовании:** "Как реализовать очередь с ограниченным размером (circular buffer)?"
- **Практика:** Реализовать класс `CircularQueue` с фиксированным размером

№ 1.3.2.3 — Deque (двусторонняя очередь)

- **Что нужно знать:** операции `push_front`, `push_back`, `pop_front`, `pop_back`
- **Вопросы на собеседовании:** "Когда использовать deque вместо vector?"
- **Практика:** Написать бенчмарк для сравнения `std::deque` и `std::vector` при вставке в начало

№ 1.3.2.4 — Применение

- **Что нужно знать:** BFS (обход графа), алгоритм очереди (FIFO), пулы потоков (thread pools)
- **Вопросы на собеседовании:** "Как используется очередь в BFS?"
- **Практика:** Реализовать BFS с использованием `std::queue`

№ 1.3.2.5 — Priority Queue (очередь с приоритетом)

- **Что нужно знать:** `std::priority_queue`, куча (heap) как основа
- **Вопросы на собеседовании:** "Как работает очередь с приоритетом? Какая структура данных лежит в основе?"
- **Практика:** Реализовать приоритетную очередь на куче (binary heap)

1.3.3. Экспертный уровень (Senior / Staff Level)

№ 1.3.3.1 — Lock-free стек/очередь

- **Что нужно знать:** реализация без мьютексов с использованием атомарных операций (CAS)
- **Вопросы на собеседовании:** "Как реализовать потокобезопасную очередь без блокировок?"
- **Практика:** Изучить алгоритм lock-free очереди (Michael-Scott) и реализовать его

№ 1.3.3.2 — Распределенные очереди

- **Что нужно знать:** Kafka, RabbitMQ (партиционирование, оффсеты)
- **Вопросы на собеседовании:** "Как устроена очередь в Apache Kafka?"
- **Практика:** Написать producer/consumer для Kafka на C++

№ 1.3.3.3 — Очереди с постоянной памятью

- **Что нужно знать:** хранение очереди на диске (например, для обработки больших данных)
- **Вопросы на собеседовании:** "Как реализовать очередь, которая не помещается в оперативную память?"
- **Практика:** Реализовать очередь с использованием mmap для хранения на диске

№ 1.3.3.4 — Блокирующие и неблокирующие очереди

- **Что нужно знать:** условие для push/pop с ожиданием
- **Вопросы на собеседовании:** "Как реализовать producer-consumer с блокировкой, когда очередь пуста или полна?"
- **Практика:** Реализовать блокирующую очередь на C++ с `std::mutex` и `std::condition_variable`

1.3.4. Практические задания (сквозные)

№ 1.3.4.1

- **Задание:** Реализовать алгоритм "сортировка стека" с использованием дополнительного стека
- **Что проверяет:** Понимание работы стека и алгоритмов

№ 1.3.4.2

- **Задание:** Реализовать очередь с помощью двух стеков (амортизированная $O(1)$)
- **Что проверяет:** Понимание амортизированной сложности

№ 1.3.4.3

- **Задание:** Реализовать алгоритм "обход бинарного дерева" с использованием стека (итеративно)
- **Что проверяет:** Применение стека для DFS

№ 1.3.4.4

- **Задание:** Реализовать пул потоков с использованием блокирующей очереди
- **Что проверяет:** Практика многопоточности

№ 1.3.4.5

- **Задание:** Написать парсер обратной польской записи (RPN) с использованием стека
 - **Что проверяет:** Классическая задача на стек
-

РАЗДЕЛ 1.4: ХЕШ-ТАБЛИЦЫ (JUNIOR → SENIOR)

1.4.1. Основы хеш-таблиц (Junior Level)

№ 1.4.1.1 — Принцип работы

- **Что нужно знать:** хеш-функция преобразует ключ в индекс, сохранение пары (ключ, значение)
- **Вопросы на собеседовании:** "Как работает хеш-таблица?"
- **Практика:** Реализовать простую хеш-таблицу с цепочками

№ 1.4.1.2 — Хеш-функция

- **Что нужно знать:** свойства (детерминированность, равномерность, быстрота вычисления)
- **Вопросы на собеседовании:** "Что такое хорошая хеш-функция?"
- **Практика:** Реализовать хеш-функцию для строк (полиномиальную)

№ 1.4.1.3 — Коллизии

- **Что нужно знать:** что это, методы разрешения (цепочки, открытая адресация)
- **Вопросы на собеседовании:** "Что такое коллизия в хеш-таблице? Как ее разрешить?"
- **Практика:** Реализовать две хеш-таблицы: с цепочками и с открытой адресацией

№ 1.4.1.4 — Метод цепочек

- **Что нужно знать:** каждый элемент — список, хранение нескольких элементов в одном индексе
- **Вопросы на собеседовании:** "Как работает метод цепочек в хеш-таблице?"
- **Практика:** Реализовать хеш-таблицу с цепочками на основе `std::list`

№ 1.4.1.5 — Открытая адресация

- **Что нужно знать:** линейное, квадратичное, двойное хеширование

- **Вопросы на собеседовании:** "В чем отличие линейного пробирования от квадратичного?"
- **Практика:** Реализовать хеш-таблицу с открытой адресацией и линейным пробированием

№ 1.4.1.6 — Load Factor (коэффициент загрузки)

- **Что нужно знать:** отношение элементов к размеру, при превышении — рехэширование
 - **Вопросы на собеседовании:** "Что такое load factor? Что происходит при его превышении?"
 - **Практика:** Реализовать рехэширование при превышении load factor = 0.75
-

1.4.2. Продвинутое хеш-таблицы (Middle Level)

№ 1.4.2.1 — std::unordered_map (C++)

- **Что нужно знать:** внутреннее устройство (корзины, хеш-функция), настройка
- **Вопросы на собеседовании:** "Как работает std::unordered_map? Чем отличается от std::map?"
- **Практика:** Написать бенчмарк сравнения std::map и std::unordered_map

№ 1.4.2.2 — Dictionary<TKey,TValue> (C#)

- **Что нужно знать:** устройство, хеш-коды, пересоздание при расширении
- **Вопросы на собеседовании:** "Как устроен Dictionary в C#?"
- **Практика:** Проанализировать через ILSpy код Dictionary

№ 1.4.2.3 — Идеальное хеширование

- **Что нужно знать:** FKS (Fredman-Komlós-Szemerédi) — статическая хеш-таблица без коллизий
- **Вопросы на собеседовании:** "Что такое идеальное хеширование и где оно применяется?"
- **Практика:** Изучить алгоритм FKS и реализовать его упрощенную версию

№ 1.4.2.4 — Устойчивость к атакам

- **Что нужно знать:** защита от коллизий (SipHash), хеширование с солью
- **Вопросы на собеседовании:** "Как защитить хеш-таблицу от атак на коллизии?"
- **Практика:** Изучить SipHash и понять, почему он используется в Python

№ 1.4.2.5 — Хеширование на основе дерева

- **Что нужно знать:** Java TreeMap vs HashMap, когда использовать
- **Вопросы на собеседовании:** "Когда использовать TreeMap, а когда HashMap?"
- **Практика:** Сравнить производительность и объяснить trade-offs

№ 1.4.2.6 — Concurrent HashMap

- **Что нужно знать:** потокобезопасные хеш-таблицы (ConcurrentHashMap в Java, ConcurrentDictionary в C#)
- **Вопросы на собеседовании:** "Как реализовать потокобезопасную хеш-таблицу?"
- **Практика:** Изучить, как работает ConcurrentDictionary в C#

1.4.3. Экспертный уровень (Senior / Staff Level)

№ 1.4.3.1 — Lock-free хеш-таблицы

- **Что нужно знать:** реализация без блокировок, использование CAS
- **Вопросы на собеседовании:** "Как реализовать хеш-таблицу без блокировок?"
- **Практика:** Изучить lock-free хеш-таблицу (например, в Java ConcurrentHashMap)

№ 1.4.3.2 — Cuckoo Hashing

- **Что нужно знать:** хеширование с двумя хеш-функциями и перемещением элементов
- **Вопросы на собеседовании:** "Что такое Cuckoo Hashing?"
- **Практика:** Реализовать Cuckoo Hashing

№ 1.4.3.3 — Robin Hood Hashing

- **Что нужно знать:** уменьшение максимального смещения в открытой адресации
- **Вопросы на собеседовании:** "В чем преимущество Robin Hood Hashing?"
- **Практика:** Реализовать Robin Hood Hashing и сравнить с линейным пробированием

№ 1.4.3.4 — Persistent Hash Map

- **Что нужно знать:** хеш-таблицы с неизменяемостью (immutable), использующие структурный шеринг
- **Вопросы на собеседовании:** "Как реализовать immutable хеш-таблицу с эффективными обновлениями?"
- **Практика:** Изучить persistent hash map (например, в Clojure)

№ 1.4.3.5 — Хеширование для распределенных систем

- **Что нужно знать:** Consistent Hashing (кольцевое хеширование)
- **Вопросы на собеседовании:** "Как работает Consistent Hashing и зачем он нужен?"
- **Практика:** Реализовать Consistent Hashing для распределенного кеша

1.4.4. Практические задания (сквозные)

№ 1.4.4.1

- **Задание:** Реализовать свою хеш-таблицу на C++ с методом цепочек и открытой адресацией
- **Что проверяет:** Понимание всех аспектов хеш-таблиц

№ 1.4.4.2

- **Задание:** Реализовать LRU Cache с использованием хеш-таблицы и двусвязного списка
- **Что проверяет:** Комбинация структур данных

№ 1.4.4.3

- **Задание:** Реализовать счетчик частоты слов в тексте с использованием хеш-таблицы
- **Что проверяет:** Практическое применение

№ 1.4.4.4

- **Задание:** Написать программу для поиска дубликатов в массиве за $O(n)$ с использованием хеш-таблицы
- **Что проверяет:** Эффективное использование хешей

№ 1.4.4.5

- **Задание:** Реализовать Consistent Hashing для шардирования данных между 3 нодами
 - **Что проверяет:** Распределенные системы
-

РАЗДЕЛ 1.5: ДЕРЕВЬЯ (JUNIOR → SENIOR)

1.5.1. Бинарное дерево поиска (Junior Level)

№ 1.5.1.1 — Определение

- **Что нужно знать:** каждый узел имеет не более двух потомков (левый < родитель < правый)
- **Вопросы на собеседовании:** "Что такое бинарное дерево поиска?"
- **Практика:** Реализовать узел Node с left, right, value

№ 1.5.1.2 — Вставка

- **Что нужно знать:** рекурсивная, сравнение с корнем, вставка в левое/правое поддерево- **Вопросы на собеседовании:** "Как вставить элемент в BST?"
- **Практика:** Реализовать метод insert

№ 1.5.1.3 — Поиск

- **Что нужно знать:** рекурсивный поиск, сравнение, движение влево/вправо
- **Вопросы на собеседовании:** "Как найти элемент в BST?"
- **Практика:** Реализовать метод search

№ 1.5.1.4 — Удаление

- **Что нужно знать:** три случая (лист, один ребенок, два ребенка — находит минимальный в правом поддереве)
- **Вопросы на собеседовании:** "Как удалить узел из BST?"
- **Практика:** Реализовать метод remove

№ 1.5.1.5 — Обходы

- **Что нужно знать:** Inorder (слева → корень → справа) — дает отсортированный порядок, Preorder (корень → слева → справа), Postorder (слева → справа → корень)
- **Вопросы на собеседовании:** "Какие есть способы обхода дерева?"
- **Практика:** Реализовать рекурсивные обходы и итеративные (со стеком)

№ 1.5.1.6 — Высота дерева

- **Что нужно знать:** глубина, максимальная высота $O(n)$ в несбалансированном случае
- **Вопросы на собеседовании:** "Какая может быть высота BST в худшем случае?"
- **Практика:** Написать функцию для вычисления высоты дерева

№ 1.5.1.7 — Минимум/максимум

- **Что нужно знать:** самый левый/правый узел
- **Вопросы на собеседовании:** "Как найти минимальный элемент в BST?"
- **Практика:** Реализовать методы `find_min` и `find_max`

1.5.2. Сбалансированные деревья (Middle Level)

№ 1.5.2.1 — Проблема вырожденного дерева

- **Что нужно знать:** BST превращается в список, высота $O(n)$
- **Вопросы на собеседовании:** "Почему BST может работать медленно?"
- **Практика:** Построить дерево из отсортированных данных и показать, как оно вырождается

№ 1.5.2.2 — AVL-деревья

- **Что нужно знать:** фактор баланса (-1, 0, 1), вращения (LL, RR, LR, RL)
- **Вопросы на собеседовании:** "Что такое AVL-дерево? Как оно балансируется?"
- **Практика:** Реализовать вставку в AVL-дерево с вращениями

№ 1.5.2.3 — Красно-черные деревья

- **Что нужно знать:** свойства (узлы черные/красные, 5 свойств), вращения, перекраска
- **Вопросы на собеседовании:** "В чем отличие красно-черного дерева от AVL?"
- **Практика:** Реализовать красно-черное дерево с вставкой

№ 1.5.2.4 — B-Tree и B+Tree

- **Что нужно знать:** структура для баз данных, узлы с несколькими ключами, степень дерева
- **Вопросы на собеседовании:** "Как устроен B-Tree? Почему он используется в базах данных?"
- **Практика:** Реализовать упрощенный B-Tree для файловой системы

№ 1.5.2.5 — `std::map` и `std::set`

- **Что нужно знать:** в STL реализованы как красно-черные деревья
- **Вопросы на собеседовании:** "Какая структура данных лежит в основе `std::map`?"
- **Практика:** Проанализировать сложность операций `std::map`

№ 1.5.2.6 — Treap (декартово дерево)

- **Что нужно знать:** комбинация BST и кучи (рандомный приоритет)
- **Вопросы на собеседовании:** "Что такое Treap?"
- **Практика:** Реализовать Treap с вставкой, поиском, удалением

1.5.3. Экспертный уровень (Senior / Staff Level)

№ 1.5.3.1 — Splay-деревья

- **Что нужно знать:** доступ к последнему запрошенному элементу за $O(\log n)$ (амортизированно), splay операция
- **Вопросы на собеседовании:** "Что такое Splay-дерево и когда его использовать?"
- **Практика:** Реализовать Splay-дерево с операциями `insert`, `search`, `delete`

№ 1.5.3.2 — Фенвика дерево (BIT)

- **Что нужно знать:** для операций префиксной суммы и обновления за $O(\log n)$

- **Вопросы на собеседовании:** "Как быстро отвечать на запросы суммы на префиксе?"
- **Практика:** Реализовать Fenwick Tree для массива с обновлениями

№ 1.5.3.3 — Дерево отрезков (Segment Tree)

- **Что нужно знать:** для запросов на отрезке (сумма, минимум, максимум) с обновлениями за $O(\log n)$
- **Вопросы на собеседовании:** "Как решить задачу RMQ (Range Minimum Query)?"
- **Практика:** Реализовать Segment Tree для суммы на отрезке с ленивыми обновлениями

№ 1.5.3.4 — Trie (префиксное дерево)

- **Что нужно знать:** для быстрого поиска строк по префиксу, автодополнение
- **Вопросы на собеседовании:** "Как реализовать автодополнение для поиска?"
- **Практика:** Реализовать Trie с методами insert, search, startsWith

№ 1.5.3.5 — Суффиксное дерево / суффиксный массив

- **Что нужно знать:** для поиска подстрок в тексте за $O(m)$
- **Вопросы на собеседовании:** "Как найти все вхождения подстроки в тексте за линейное время?"
- **Практика:** Реализовать суффиксный массив и алгоритм поиска подстроки

№ 1.5.3.6 — R-tree

- **Что нужно знать:** структура для пространственных данных (географические координаты)
- **Вопросы на собеседовании:** "Какая структура данных используется для пространственного поиска?"
- **Практика:** Изучить R-tree и его применение в ГИС

1.5.4. Практические задания (сквозные)

№ 1.5.4.1

- **Задание:** Реализовать BST с вставкой, удалением, поиском, обходами

- **Что проверяет:** Базовое понимание деревьев

№ 1.5.4.2

- **Задание:** Реализовать AVL-дерево с вставкой и удалением с вращениями
- **Что проверяет:** Понимание балансировки

№ 1.5.4.3

- **Задание:** Реализовать проверку, является ли бинарное дерево BST (валидатор)
- **Что проверяет:** Проверка свойств дерева

№ 1.5.4.4

- **Задание:** Реализовать поиск LCA (Lowest Common Ancestor) в BST
- **Что проверяет:** Классическая задача на деревья

№ 1.5.4.5

- **Задание:** Реализовать сериализацию/десериализацию бинарного дерева
- **Что проверяет:** Работа с деревьями и строками

№ 1.5.4.6

- **Задание:** Реализовать алгоритм "построить дерево из обходов preorder и inorder"
- **Что проверяет:** Понимание обходов дерева

РАЗДЕЛ 1.6: ГРАФЫ (JUNIOR → SENIOR)

1.6.1. Основы графов (Junior Level)

№ 1.6.1.1 — Определение

- **Что нужно знать:** вершины (nodes) и ребра (edges), ориентированные и неориентированные графы
- **Вопросы на собеседовании:** "Что такое граф? Приведи примеры из реальной жизни"
- **Практика:** Нарисовать граф социальной сети

№ 1.6.1.2 — Представление

- **Что нужно знать:** матрица смежности ($O(V^2)$ памяти), список смежности ($O(V+E)$ памяти)
- **Вопросы на собеседовании:** "Какие есть способы представления графа в памяти?"
- **Практика:** Реализовать класс Graph с обоими представлениями

№ 1.6.1.3 — Степень вершины

- **Что нужно знать:** количество ребер, входящих/исходящих
- **Вопросы на собеседовании:** "Что такое степень вершины в ориентированном графе?"
- **Практика:** Вычислить степени всех вершин графа

№ 1.6.1.4 — Обход графа

- **Что нужно знать:** BFS (очередь) и DFS (стек/рекурсия)
- **Вопросы на собеседовании:** "В чем разница между BFS и DFS?"
- **Практика:** Реализовать BFS и DFS для графа

№ 1.6.1.5 — Поиск компонент связности

- **Что нужно знать:** BFS/DFS для нахождения всех компонент в неориентированном графе
- **Вопросы на собеседовании:** "Как найти все компоненты связности в графе?"
- **Практика:** Написать программу для выделения компонент связности

№ 1.6.1.6 — Топологическая сортировка

- **Что нужно знать:** для ациклических ориентированных графов (DAG)
- **Вопросы на собеседовании:** "Когда используется топологическая сортировка?"
- **Практика:** Реализовать топологическую сортировку (Kahn и DFS)

1.6.2. Кратчайшие пути и алгоритмы (Middle Level)

№ 1.6.2.1 — Алгоритм Дейкстры

- **Что нужно знать:** поиск кратчайшего пути от одной вершины, работает с неотрицательными весами, $O(E \log V)$

- **Вопросы на собеседовании:** "Как найти кратчайший путь в графе с неотрицательными весами?"
- **Практика:** Реализовать алгоритм Дейкстры с приоритетной очередью

№ 1.6.2.2 — Алгоритм Форда-Беллмана

- **Что нужно знать:** работает с отрицательными ребрами, детектирует отрицательные циклы, $O(V \cdot E)$
- **Вопросы на собеседовании:** "Как найти кратчайший путь, если есть отрицательные ребра?"
- **Практика:** Реализовать алгоритм Форда-Беллмана

№ 1.6.2.3 — Алгоритм Флойда-Уоршелла

- **Что нужно знать:** поиск кратчайших путей между всеми парами вершин, $O(V^3)$
- **Вопросы на собеседовании:** "Как найти кратчайшие пути между всеми парами вершин?"
- **Практика:** Реализовать алгоритм Флойда-Уоршелла

№ 1.6.2.4 — A алгоритм*

- **Что нужно знать:** эвристический поиск с эвристикой (например, Manhattan distance), используется в играх и картах
- **Вопросы на собеседовании:** "В чем отличие A* от Дейкстры?"
- **Практика:** Реализовать A* для поиска пути в лабиринте

№ 1.6.2.5 — Задача о максимальном потоке

- **Что нужно знать:** алгоритм Форда-Фалкерсона (поиск увеличивающего пути)
- **Вопросы на собеседовании:** "Как найти максимальный поток в сети?"
- **Практика:** Реализовать алгоритм Эдмондса-Карпа (BFS для поиска пути)

№ 1.6.2.6 — Минимальное остовное дерево

- **Что нужно знать:** алгоритм Прима (с приоритетной очередью), алгоритм Краскала (DSU)
- **Вопросы на собеседовании:** "Как построить минимальное остовное дерево?"
- **Практика:** Реализовать алгоритмы Прима и Краскала

1.6.3. Экспертный уровень (Senior / Staff Level)

№ 1.6.3.1 — Алгоритм Тарьяна

- **Что нужно знать:** поиск мостов (bridges) и точек сочленения (articulation points) за $O(V+E)$
- **Вопросы на собеседовании:** "Как найти мосты в графе?"
- **Практика:** Реализовать алгоритм Тарьяна для поиска мостов

№ 1.6.3.2 — Сильно связанные компоненты

- **Что нужно знать:** алгоритм Косарайю (два прохода DFS) или Тарьяна
- **Вопросы на собеседовании:** "Как найти сильно связанные компоненты в ориентированном графе?"
- **Практика:** Реализовать алгоритм Косарайю

№ 1.6.3.3 — 2-SAT

- **Что нужно знать:** выполнимость булевых формул в конъюнктивной нормальной форме (2-CNF)
- **Вопросы на собеседовании:** "Как решить задачу 2-SAT с помощью графов?"
- **Практика:** Реализовать 2-SAT с использованием импликационного графа

№ 1.6.3.4 — Поток в сетях с минимальной стоимостью

- **Что нужно знать:** алгоритм последовательного кратчайшего пути, алгоритм Беллмана-Форда на остаточной сети
- **Вопросы на собеседовании:** "Как решить задачу о потоке с минимальной стоимостью?"
- **Практика:** Реализовать алгоритм для потока минимальной стоимости

№ 1.6.3.5 — Распределенные графовые алгоритмы

- **Что нужно знать:** PageRank (Google), алгоритмы для больших графов (GraphX, Pregel)
 - **Вопросы на собеседовании:** "Как работает PageRank?"
 - **Практика:** Реализовать PageRank на C++ для графа из 1 млн вершин
-

1.6.4. Практические задания (сквозные)

№ 1.6.4.1

- **Задание:** Реализовать алгоритм для поиска кратчайшего пути в лабиринте (матрица)
- **Что проверяет:** Понимание BFS/DFS

№ 1.6.4.2

- **Задание:** Реализовать систему рекомендаций на основе графов (друзья друзей)
- **Что проверяет:** Применение графовых алгоритмов

№ 1.6.4.3

- **Задание:** Реализовать алгоритм для раскраски графа
- **Что проверяет:** Знание жадных алгоритмов

№ 1.6.4.4

- **Задание:** Реализовать алгоритм для поиска эйлерова цикла (Hierholzer)
- **Что проверяет:** Сложные графовые алгоритмы

№ 1.6.4.5

- **Задание:** Реализовать алгоритм для топологической сортировки и детекции цикла в графе зависимостей (пакетный менеджер)
- **Что проверяет:** Практическое применение

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Массивы: объявление, обход, бинарный поиск

Строки: C-style, std::string, поиск подстроки (наивный)

Списки: односвязный, двусвязный, вставка/удаление/обход

Стек/очередь: реализация на массиве/списке, применение

Хеш-таблицы: принцип работы, коллизии (цепочки)

Деревья: BST, вставка, удаление, обходы

Графы: представление, BFS, DFS

Сложность: O-большое ($O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$)

Middle Level (должен знать 100%)

Алгоритмы на массивах: два указателя, скользящее окно, prefix sum

КМП, алгоритм Бойера-Мура (понимание)

Хеш-таблицы: открытая адресация, load factor, рехэширование

AVL-деревья, красно-черные деревья (понимание)

Алгоритм Дейкстры, Форда-Беллмана

Алгоритм Прима/Краскала (минимальное остовное дерево)

Топологическая сортировка

Амортизированный анализ (push_back в векторе)

Senior Level (должен знать 100%)

Все структуры данных реализованы с нуля (C++/C#)

Алгоритм Тарьяна (мосты, точки сочленения)

Алгоритм Косараяю (сильно связанные компоненты)

Максимальный поток (Эдмондс-Карп, Форд-Фалкерсон)

Tree, Segment Tree, Fenwick Tree

Trie, суффиксный массив

B-Tree, B+Tree (для БД)

Consistent Hashing

A*, 2-SAT

Lock-free структуры (stack, queue)

🔨 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 1

Задание: Реализовать поисковую систему (inverted index) для небольшого набора документов.

Шаги:

1. Чтение документов из папки

2. Токенизация текста (разбиение на слова)
3. Построение инвертированного индекса (слово $\rightarrow$ список документов)
4. Реализация поиска по одному слову (хеш-таблица)
5. Реализация поиска по фразе с использованием алгоритмов на строках
6. Реализация ранжирования результатов (TF-IDF)
7. Оптимизация: сортировка результатов, кэширование
8. Добавление поддержки многопоточности для обработки больших документов

Проверка:

Программа должна отвечать на запросы за $O(\log n)$ или $O(1)$ с использованием хеш-таблиц и деревьев.